

Design and Data Structure of Fully Adaptive, Multigrid, Finite-Element Software

Maria-Cecilia Rivara

Catholic University, Leuven, Belgium

The design and data structure of fully adaptive, multigrid, finite-element software are presented and discussed. Adaptivity is taken in a general sense to mean that optionally, sequences of finite-element problems can be constructed in the self-adaptive or quasi-interactive mode. The software combines linear, triangular, finite-element spaces; a posteriori error estimates; adaptivity of the mesh, conforming mesh-refinement algorithms for triangulations, and a full multigrid method for solving the linear systems. A molecular List data structure, which requires a minimum amount of storage, is introduced and justified. Indeed no more storage than that required to store the nonzero elements of the sequences of the (non-symmetric) stiffness matrices is needed. As a result, an experimental, general, and flexible software is obtained. Numerical experiments performed with singular problems have shown that, in the self-adaptive mode, the numerical rate of convergence obtained approaches the maximum theoretical rate of convergence predicted for linear elements.

Categories and Subject Descriptors: D.2.0 [Software Engineering]: General; F.2.1 [Analysis of Algorithms and Problem Complexity]: Numerical Algorithms and Problems—*computation on matrices*; G.1.3 [Numerical Analysis]: Numerical Linear Algebra—*linear systems (direct and iterative methods)*; *sparse and very large systems*; G.1.8 [Numerical Analysis]: Partial Differential Equations—*elliptic equations*; *finite-element methods*; K.6.3 [Management of Computing and Information Systems]: Software Management—*software development*

General Terms: Algorithms, Design Management

Additional Key Words and Phrases: Adaptive mesh refinement, multigrid methods

1. INTRODUCTION

In this work, the design and data structure of fully adaptive, multigrid, finite-element software, which exploit the intrinsic features of the h version of the finite-element method over conforming triangulations, are presented and discussed. This *software* constitutes a prototype of a general and useful tool for solving partial differential equations.

Instead of considering a fixed, discretized approximation of the continuous boundary value problem, our approach centers on the construction of an adequate sequence of discretized problems that properly approximate the continuous problem. To develop such software, the overall solution process, the data structure, and the different techniques combined in the software have to be carefully

Authors' present address: Department of Mathematics, University of Chile, Casilla 5272 Correo 3, Santiago, Chile.

Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the ACM copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Association for Computing Machinery. To copy otherwise, or to republish, requires a fee and/or specific permission.

© 1984 ACM 0098-3500/84/0900-0242 \$00.75

ACM Transactions on Mathematical Software, Vol. 10, No. 3, September 1984, pages 242-264.

considered. Thus, for instance, in order to manage singular solutions, adequate algorithms that allow the selective refinement of conforming triangulations have to be included. The available current numerical solution should be used to define the following discretized problem. Moreover, in this context, full multigrid methods appear as the most appropriate techniques for solving the sequence of sparse systems of equations generated.

The data structure to be used with such a complex design is certainly critical. In fact, the lack of good data structures has been frequently used as an argument against fully adaptive and/or multigrid software. In this paper we show that such a critical problem can be solved effectively by introducing the concept of molecular structure associated with any conforming triangulation. The iterative solvers for sparse systems of equations [24] give us the basis for designing the data structure. In effect, the compact row-wise representation of sparse matrices really contains a lot of information about the grid. By managing the diagonal elements in a separate vector and by introducing an order over the data, no more storage space than that required for the nonzero elements of the sequence of stiffness matrices is needed.

An experimental version of this software has been reasonably implemented. Furthermore, we suspect that the complexity of implementation of our approach is no greater than that of the adaptive and/or multigrid finite-element software so far developed.

2. STATIC VERSUS DYNAMIC APPROACH

The development of complex software, such as that related to finite-element methods for solving elliptic boundary value problems, has been greatly influenced by two factors: first by the static approach considered, namely, where the continuous problem is approximated by a fixed discretized problem, and second, by the state of the art of the related techniques available for designing the software. In an interactive way, the static approach has necessarily influenced the development of these techniques.

Direct methods for solving sparse systems of equations associated with the finite-element method are based on the static structure of the discretized problem (graph properties of the grid). For more than 10 years, an enormous effort has been devoted to studying the best ordering of the nodes in order to get minimum banded sparse matrices [10]. For most of the classical finite-element software developed so far, the three separate steps for solving partial differential equations or related physical problems also reflect this tendency (see, e.g., [9]).

On the other hand, the static discretized approach considered has, in a certain sense, discouraged the development of other techniques. That happened with iterative methods for solving sparse systems of equations, which although used at the beginning, have been lately discarded [24]. Uncertainties about selection of the acceleration parameters and termination of the iteration process have also contributed to discouraging the use of iterative methods.

However, parallel to the interaction discussed above, some interesting and related techniques have also been developed and justified during recent years. Features of these have induced a change in the practical approach so far accepted as the most feasible and appropriate with finite-element software. From the point

of view of the implementation of the h version of the finite-element method, the two most interesting and promising of these techniques are certainly adaptivity of the grid [1-4, 5, 7, 16, 18-21, 23, 25] and the multigrid or multilevel methods for solving sparse systems of equations associated with partial differential equations [5-8, 11, 12, 15, 22]. Both techniques consider the problem in a dynamic way; that is, they are based on the fact that the continuous boundary value problem can be approximated by a sequence of discretized problems which properly use the ellipticity of the boundary value problem.

Multigrid methods use iterative solvers of equations to smooth either the solution or related residual functions over coarser and coarser grids [5-8, 11, 12, 15, 22]. This proceeds only until the functions become smooth, namely, while the iterative methods are rapidly convergent.

Adaptivity of the grid is a desirable feature since it relieves the user of critical decisions and allows use of all the flexibility of the finite-element method for getting a minimum number of nodes (quasi-optimal discretization) [1-4]. Notice that in this sense the generation of the discretization should not be a first separate step of the software, but a dynamic adaptive process.

In order to deal with singular solutions, the capability for managing local refinement of the discretization is indispensable, and mesh-refinement algorithms that maintain the nondegeneracy of the elements and the conformity and smoothness of the grid (the transition between small and large elements should be smooth) are certainly desirable (see, e.g., [4, 7]).

It is clear that the techniques chosen to satisfy the different goals proposed should also fit together in an efficient way. It has frequently happened that potentially good techniques have been discarded because they seem not to fit properly with the other related techniques needed in the software. As a typical example, fully adaptive discretization has been repeatedly rejected because of the lack of good mesh-refinement algorithms and the great deal of storage and bookkeeping they apparently need (see, e.g., [8]). One of the most critical decisions with this kind of software is the definition of the data structure to be used to construct the sequence of discretized problems and to manage the multigrid method. This definition must necessarily consider the different algorithms to be used with it.

3. SURVEY ON ADAPTIVE MULTIGRID SOFTWARE

In recent years, much work has been done on the implementation of the ideas discussed in the preceding section. With the already implemented and available codes including some of the features discussed above, we should mention the FEARS package of Babuska and Rheinboldt [4, 16, 25], which uses rectangular elements and self-adaptive refinement of the meshes; the PLTMG package of Bank and Sherman [5, 7], which includes self-adaptive refinement over triangulations and a multigrid algorithm; and the TWODEPEP package of Sewell [21], which uses a certain kind of user-guided refinement over triangulations. We should also mention the thesis of Weiser [23], where self-adaptive refinement of rectangular grids is used in the context of a combined h - p version of the finite-element method.

The self-adaptive aspect has been developed and justified by Babuska and Rheinboldt through several articles (see, e.g., [1-4]). In practice, in the FEARS

package, adaptivity over rectangular meshes has been managed by combining the computation of local error estimators over individual elements with a mesh-equilibration procedure of the error. Reported numerical results have shown that optimal rates of convergence have been achieved and good error estimates computed (see, e.g., [4]). The same ideas have been applied in the PLTMG package of Bank and Sherman [5, 7] over triangular meshes. However, as far as we know, practical evaluation of the quality of their error estimators has not been published yet.

The mesh-refinement algorithms used in the FEARS and PLTMG packages have been based on the construction of elements (rectangles and triangles, respectively) similar to those of the initial mesh. In this way, a set of nonconforming nodes is obtained when local refinement is used. For solving this problem, Bank and Sherman [7] introduce a set of "green" triangles (they join each nonconforming point with the opposite vertex of the nonconforming triangle). However, they have to force the triangulations to be smooth, and, as a consequence, the triangulations obtained are not strictly nested. Conversely, Rheinboldt and Metsztenyi [16] work with the nonconforming grids, and, in order to maintain the continuity of the solution over the nonconforming points, they impose the condition that the solution over them be equal to values interpolated from the nearby conforming points. However, the appearance of these nonconforming nodes complicates the calculation of the elementary stiffness matrices. Finally, we should mention that in the TWODEPEP package of Sewell [21], the local refinement has been managed by bisecting the selected triangles by the longest side; however, to assure conformity, each nonconforming midpoint generated is directly connected with the opposite vertex of the neighboring triangle (even if the shared side is not the longest side of the neighboring triangle). As a consequence, the nondegeneracy of the meshes cannot be assured.

In respect to the data structure, both the FEARS and PLTMG packages use a data structure based on the refinement tree associated with the generation of elements similar to those of the initial grid. Both data structures indeed use the fact that, by knowing the level of a given element and its position on the refinement tree, its geometrical characteristics can be easily reconstructed.

The software whose design and data structure are discussed throughout this paper is based on the idea that the use of fully irregular and conforming triangulations is the most flexible, simple, and appropriate way of exploiting the intrinsic features of the h version of the finite-element method. To show the feasibility of this approach, two items are keys: the availability of good mesh-refinement algorithms for the selective refinement of general conforming triangulations, and the data structure to be used to manage the dynamic construction of the meshes and the multigrid algorithm.

The first problem has been satisfactorily solved in [19] and [20], where mesh-refinement algorithms, which can be applied globally or locally for selective refinement of conforming triangulations and always produce conforming triangulations with excellent geometrical properties, have been introduced and justified.

The main goal of this work is to introduce and discuss a data structure suitable for managing both the dynamic construction of fully irregular and conforming triangulations, and an appropriate full multigrid algorithm, in the context of the

self-adaptive approach of Babuska and Rheinboldt. Furthermore, numerical results obtained with an experimental version of this software are also presented and discussed.

4. CHARACTERISTICS OF THE SOFTWARE

In this section the different techniques chosen to satisfy the goals proposed are briefly discussed, with emphasis on the new conforming mesh-refinement algorithms and the multigrid method.

Our main objective is to design a general, easy-to-use software that is able to solve a reasonably wide class of partial differential equations, over linear, triangular, finite-element spaces.

In our design, adaptivity is generally understood, in such a way, that either the software can be guided by using the knowledge the user has over the problem to construct sequences of quasi-optimal meshes, or the software self adaptively constructs such sequences by using local error indicators to predict which elements should be refined in the next grid. The approach of Babuska and Rheinboldt has been used to develop local error estimators over triangles [18].

To clarify the design of our software, it is convenient to consider the generation of each new mesh in two separate steps: First, a refinement submesh is determined (a set of triangles to be refined in the next triangulation) and, second, a conforming mesh-refinement algorithm produces an acceptable conforming triangulation. The first step corresponds to the adaptive process, which can be either self- or user-adaptive.

4.1. Adaptive Refinement

As stated above, this subsection refers to the strategy to be followed to construct automatically a refinement submesh (a set of triangles to be refined in the next triangulation) after a computed solution has been obtained over the current triangulation. Such strategy can be either user- or self-adaptive.

Because of the useful characteristics of the conforming mesh-refinement algorithms (see Section 4.2), the user can easily define dynamic refinement subregions throughout the different levels according to the knowledge he or she has over the problem. The possibility of performing uniform or global refinement has been also considered.

The self-adaptive strategy is essentially that of Babuska and Rheinboldt (see, e.g., [3, 4]). It basically consists of the evaluation of local error estimators on individual elements, which are used to forecast new error estimators for the refined elements. The error is (asymptotically) optimal for the degrees of freedom used if all the indicators are essentially equal [4], and hence the refinement submesh is determined to achieve a more equal distribution of the local error indicators.

The simple heuristic scheme that we have used to forecast the new error estimators is based on the assumption that the error indicators have locally a similar behavior between grids $k - 1$ and k , and grids k and $k + 1$. Thus if any triangle Δ , with error indicator η_i was generated by refining a triangle with value η_i^{old} , then the new value η_i^{new} obtained after bisecting Δ , will be approximately,

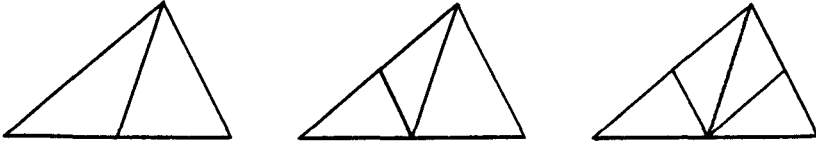


Fig. 1. Possible cases of refined triangles.

either

$$\eta_i^{\text{new}} = \frac{\eta_i^2}{\eta_i^{\text{old}}} \quad \text{or} \quad \eta_i^{\text{new}} = \left(\frac{\eta_i^3}{\eta_i^{\text{old}}} \right)^{1/2}$$

depending on whether the area of triangle Δ_i is one-half or one-fourth of the area of its predecessor (see also Section 4.2).

The algorithm used to construct the refinement submesh is essentially the same as that given in [3].

4.2. Conforming Mesh-Refinement Algorithms

Conforming mesh-refinement algorithms for triangulations have been extensively discussed in [19] and [20]. They are based on the bisection of triangles by the longest side and their useful features can be summarized as follows:

For any conforming triangulation τ (where the intersection of two non-disjoint, nonidentical triangles is either a common vertex or a common side) and any refinement submesh $V \subset \tau$, these algorithms produce a new conforming triangulation τ^* with the following characteristics:

- (i) All elements of V are refined in τ^* .
- (ii) τ^* is nested in τ in such a way that each refined triangle is embedded in one triangle of τ in one of the three ways shown in Figure 1.
- (iii) Triangulation τ^* is nondegenerate; namely, the interior angles of all triangles of τ^* are guaranteed to be bounded away from 0.
- (iv) The transition between large and small triangles is not abrupt and, in this sense, we say that τ^* is smooth.

Two conforming mesh-refinement algorithms have been included in our design. Before describing them, the following definition needs to be introduced.

Definition. Triangle $\Delta(a, b, c)$ with vertices a, b, c is a $\frac{1}{2}$ -nonconforming triangle in triangulation τ (with the $\frac{1}{2}$ -nonconforming side ac) if there exist two triangles $\Delta(a, d, e)$ and $\Delta(e, d, c)$ in τ such that

- (i) d is the midpoint of side ac ,
- (ii) $\Delta(a, b, c) \cap \Delta(a, d, e) = \text{side } ad$,
- (iii) $\Delta(a, b, c) \cap \Delta(e, d, c) = \text{side } dc$.

To illustrate this definition, see Figure 2.

Our design considers a global conforming mesh-refinement algorithm, which, for any conforming triangulation τ , produces a totally refined and conforming

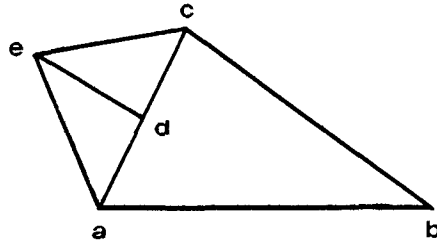


Fig. 2. A 1/2-nonconforming triangulation.

triangulation τ^* (global algorithm 2 in reference [19]) in the following way:

- (1) Bisect Δ by the longest side, for each $\Delta \in \tau$.
Let τ^* be the triangulation generated in this way.
- (2) Find S , the set of $\frac{1}{2}$ -nonconforming triangles of triangulation τ^* .
- (3) For each $\Delta \in S$, join the $\frac{1}{2}$ -nonconforming node with the opposite vertex.

The local conforming mesh-refinement algorithm considered in our software corresponds to local algorithm 2 of reference [19]. For a given conforming triangulation τ and any refinement submesh $V \subset \tau$, a new refined, conforming triangulation τ^* is generated by means of the following simple procedure:

- (1) Bisect Δ by its longest side, for all $\Delta \in V$.
Let $\tilde{\tau}_1$ be the triangulation generated in this way.
- (2) Find $R_1 \subset \tilde{\tau}_1$, the set of $\frac{1}{2}$ -nonconforming triangles of triangulation $\tilde{\tau}_1$. Set $i \leftarrow 1$.
- (3) Do for each $\Delta \in R_i$ (suppose that P is the $\frac{1}{2}$ -nonconforming node of Δ).
 - (a) Bisect Δ by its longest side. Let Q be the node generated in this way.
 - (b) If points P and Q do not coincide, then join P and Q .
- (4) Let $\tilde{\tau}_{i+1}$ be the triangulation generated in step 3.
Find $R_{i+1} \subset \tilde{\tau}_{i+1}$, the set of $\frac{1}{2}$ -nonconforming triangles of triangulation $\tilde{\tau}_{i+1}$.
- (5) If cardinality of R_{i+1} is 0, then $\tau^* = \tilde{\tau}_{i+1}$ and stop.
Otherwise set $i \leftarrow i + 1$ and go to 3.

More details concerning the performance of these algorithms, as well as some examples, can be found in [19].

4.3. The Multigrid Algorithm

The interest in multigrid methods is primarily due to the fact they are of optimal order in terms of computational work [6, 11, 12, 15, 22]. Thus, for smooth solutions and linear, triangular, finite elements in uniform grids, the work required to compute an approximation to the finite-element solution over triangulation τ_k (which satisfies the condition that the relative error in energy norm is $O(N_k^{-1/2})$), is proportional to N_k , where N_k is the number of unknowns in the linear system. Notice that in this way one obtains the maximum number of significant digits that the finite element solution has as an approximation to the exact solution of the continuous boundary value problem.

To solve the sequence of sparse linear systems generated in the self-adaptive or quasi-interactive mode, our software uses a basic full multigrid algorithm

based on a simple V-cycle iteration to solve each finite-element system. For a discussion of these concepts see, e.g., [11], [12], and [22].

In a schematic way, suppose that throughout the adaptive procedure, the software has generated a sequence of nested triangulations $\tau_1, \tau_2, \dots, \tau_k$ of the region Ω and the corresponding sequence of stiffness matrices $A_1, A_2, \dots, A_k$. Then, in order to solve the k th finite-element system,

$$A_k Z_k = b_k \quad \text{over } \tau_k,$$

the following V-cycle iteration has been used in our design:

- A. Set $\epsilon_k \leftarrow Z_k, r_k \leftarrow b_k$.
- B. Do steps E1–B3 for $j = k, k-1, \dots, k_{\min} + 1$:
 - B1. Iterate over $A_j, \epsilon_j = r_j$ (p sweeps). Let ϵ_j^1 be the approximation obtained.
 - B2. Construct the residual $\tilde{r}_j = r_j - A_j \epsilon_j^1$.
 - B3. Transfer the residual $\tilde{r}_j$ to τ_{j-1} . Let r_{j-1} be the transferred residual.
- C. Solve exactly the system $A_{k_{\min}} b_{k_{\min}} = r_{k_{\min}}$.
- D. Set $\epsilon_{k_{\min}}^1 = \epsilon_{k_{\min}}$.
- E. Do steps E1–E3 for $j = k_{\min} + 1, \dots, k-1$:
 - E1. Transfer ϵ_j^1 to τ_{j+1} . Let $\tilde{\epsilon}_{j+1}^1$ be the transferred vector.
 - E2. Correct $\epsilon_{j+1}^2 = \epsilon_{j+1}^1 + \tilde{\epsilon}_{j+1}^1$.
 - E3. Iterate over $A_{j+1} \epsilon_{j+1} = r_{j+1}$ (q sweeps) with the initial estimation ϵ_{j+1}^2 . Let ϵ_{j+1}^1 be the approximation obtained.
- F. Correct $Z_k = Z_k + \epsilon_k^1$.

Our software uses a Gauss–Seidel method for performing the p and q iterations of steps B1 and E3, as well as to solve the linear system at level $k_{\min}$.

Since in presence of highly singular solutions, the self-adaptive procedure can generate a large sequence of finite-element spaces of slowly increasing dimension (see, e.g., [7]), the efficiency of the classical full multigrid method can be seriously affected. To manage this problem, Bank and Sherman [5, 7] used a level compression procedure that eliminates some intermediate grids. In contrast, we have developed a modified full multigrid algorithm with the following two main features:

(i) The finite-element problem at level $k+1$ is solved by using a dynamic multigrid iteration, only if the number of nodes adaptively created between levels k and $k+1$, is big enough with respect to the number of vertices of triangulation k . Otherwise, a fixed number of iterations (over a local subproblem) are performed in order to obtain an approximation of the finite-element solution at level $k+1$.

(ii) The dynamic multigrid iteration essentially corresponds to a modified V-cycle of the classical multigrid method, where local or global transfer of the residuals and iterative sweeps are performed between consecutive grids, depending on the number of nodes adaptively created between both levels. Notice that, in this way, more iterative sweeps are performed around the singularities.

5. THE DATA STRUCTURE

Ideally, the data structure should efficiently manage all the algorithms described in Section 4 with a minimum amount of storage.

The adaptive and/or multigrid finite-element software so far developed has used a data structure based on the refinement tree associated with the generation

of elements similar to those of the initial grid [5, 6, 7, 16, 25]. The main advantages of this kind of data structure have been conveniently pointed out by Rheinboldt and Mesztenyi [16].

For multigrid methods whose mesh-refinement algorithms are based on the construction of elements similar to those of the initial grid, such a data structure is also appropriate, since by knowing the level of a given element and its position on the refinement tree, its geometrical characteristics can be reconstructed. This fact has also been used as an argument against a more general fully adaptive approach [8].

However, such a structure is not the most efficient one for fully adaptive, multigrid, finite-element software, nor for mesh-refinement algorithms based on the bisection of triangles by the longest side. On the contrary, we shall show that it is possible to construct a data structure that uses less storage than those of [5], [7], [16] and, [25] with comparable complexity of implementation.

First, it is important to remember that iterative solvers of equations are central to the development of multigrid software. Moreover, it is widely known that such solvers use the minimum possible machine storage for solving the linear systems associated with partial differential equations [13, 24].

The most used and economic way of storing a sparse matrix is a compact row-wise representation of the nonzero elements [13]. For a nonsymmetric stiffness matrix, all the nonzero values in each row are stored in a contiguous block of data in a real-valued array A . Two integer vectors IA and JA are additionally needed. The first, IA , defines the positions at which the information corresponding to each row starts in the vectors JA and A . The second, JA , contains the column numbers associated with the elements of A ; thus $JA(K)$ is the column number for the value $A(K)$.

At this point, it is worth noting that, for linear, triangular, finite elements, the structure really defines the grid, since the vectors IA and JA can also represent the set of molecules of the corresponding triangulation. In effect, $IA(j)$ contains a pointer to the first position at which the information about node j is stored, and the elements $JA(IA(j))$, $JA(IA(j) + 1)$, $\dots$, $JA(IA(j + 1) - 1)$ define the neighboring nodes of node j , the only exception being the corresponding diagonal element of the matrix. Thus, if the diagonal elements are managed in a separate vector, and if an order is additionally included over the molecules, all the information concerning the grid will be available, together with the stiffness matrix.

In order to manage the sequence of finite-element spaces used in the multigrid method, the data structure discussed in this section needs to be generalized.

5.1. List Structure

In this section the concept of List as a generalization of the information structure tree will be considered. For a more detailed discussion see [14].

As a data structure, a List is defined recursively as a finite sequence of atoms or Lists. Here, an atom is an element of any universe of objects that can be distinguished from a List, although, strictly speaking, an atom can be considered a particular case of List.

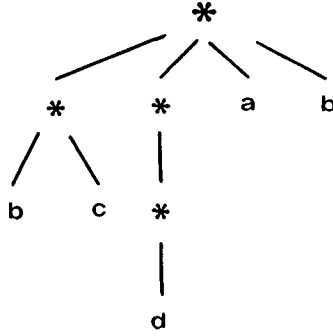


Fig. 3. Tree diagram associated with a List.

By means of an obvious notational convention involving commas and parentheses, atoms and Lists can be distinguished, and the order within a List can be adequately displayed. As an example consider $L = (L_1, L_2, a, b)$, $L_1 = (b, c)$, $L_2 = ((d))$, where L is a List with four elements with a given order. First the List L_1 , then the List L_2 , then the atom a , and finally the atom b . The List L_1 is composed of two ordered atoms, b and c . The List L_2 contains the list (d) which contains the atom d . The tree diagram of Figure 3 can be associated with this example [14], where the asterisks indicate the definition and appearance of a List as opposed to an atom.

No data are carried in the nodes for the Lists in the example considered above except the fact that they are Lists. However, it is possible to label the elements of Lists with information, as for trees and other structures [14]; e.g., $:L = (a:(b, c), d:())$ whose tree diagram is evident.

The main difference between Lists and trees is that Lists may overlap and they may even be recursive.

5.2. Basic Molecular List Structure and Conforming Triangulations

Let τ be any conforming triangulation (see Section 4.2) defined by the set $N(\tau)$ of its nodes or vertices. For each node $j \in N(\tau)$, define the integer-valued function n , in such a way, that $n(j)$ will be the number of nodes directly connected with node j . See Figure 4 where $n(7) = 5$.

Also, define the binary function $b(j)$ such that

$$b(j) = \begin{cases} 0 & \text{if } j \text{ is an interior node;} \\ 1 & \text{if } j \text{ is a boundary node.} \end{cases}$$

In Fig. 4, for instance, $b(1) = 1$, $b(7) = 0$.

The molecule j is defined as the polygonal region whose vertices are nodes of τ and whose interior only contains the node j . Thus in Fig. 4, the shaded region represents the molecule 7.

Next, for each $j \in N(\tau)$, the molecular List associated with node j will be the following:

$$M(j) = (j_1, j_2, \dots, j_{n(j)}), \quad (1)$$

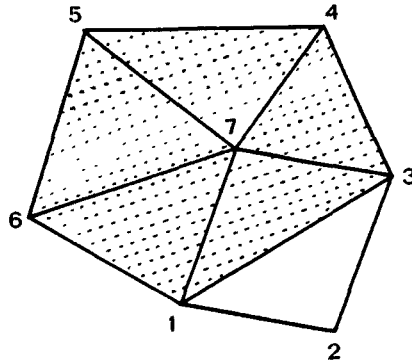


Fig. 4. A conforming triangulation.

where

- (i) $j_1, j_2, \dots, j_{n(j)}$ are the vertices of molecule j ;
- (ii) if $b(j) = 1$, then $b(j_1) = 1$ (if j is a boundary node, j_1 must also be a boundary node).
- (iii) $\text{angle } j_k j_{k+1} < \text{angle } j_k j_{k+2}$ for $k = 1, 2, \dots, n(j) - 2$, where all angles are considered in a counterclockwise sense.

To illustrate this, consider the example of Figure 4 when $M(7) = (3, 4, 5, 6, 1)$ and $M(3) = (4, 7, 1, 2)$.

The order introduced by condition (iii) will indeed be basic to searching the neighboring triangles of a given triangle. Notice also that for internal nodes, $n(j)$ different Lists can represent the molecular List structure of node j , whereas there exists only a unique List that describes the molecular List structure of a boundary node.

With the List (1) introduced above, the conforming triangulation τ can be represented by the following List:

$$L(\tau) = (M(1), M(2), \dots, M(N)), \quad (2)$$

where N is the number of elements of $N(\tau)$.

Finally, it should be pointed out that the data structure discussed here can also be used to manage the classical finite-element problem over a fully irregular fixed triangulation. Thus, if N is the number of nodes, the set of triangles can be generated in the following ordered way, which we shall call the order of the molecules:

$$\tau = \bigcup_{j=1}^N \{T(j, j_k, j_{k+1}) \times k = 1, 2, \dots, n(j) - 1; j_k, j_{k+1} \in M(j); j_k > j, j_{k+1} > j\} \cup V_j$$

where

$$V_j = \begin{cases} \{T(j, j_{n(j)}, j_1)\} & \text{if } b(j) = 0 \text{ and } j_{n(j)} > j, j_1 > j, \\ \emptyset & \text{otherwise,} \end{cases}$$

and $T(i, j, k)$ is the triangle whose vertices are the nodes i, j, k .

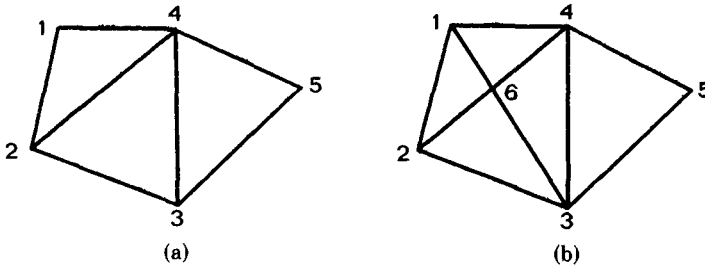


Fig. 5. Example: Triangulation (b) was obtained by refining triangulation (a).

The order of the molecules is indeed used in our experimental software each time that the triangles of a given triangulation need to be traversed, e.g., in the generation of the stiffness matrices, in the global conforming mesh-refinement algorithm, and in the computation of the local error indicators.

5.3. Molecular List Structure for Multigrid Methods

Our data structure for multigrid methods is based on the basic molecular List structure described in Section 5.2. Consequently, each grid is represented by a set of molecular Lists. However, since for highly singular solutions, the adaptive procedure can generate large sequences of only locally different triangulations (see also Section 4.3), we shall consider a compact data structure, where each molecular List appears only once. Thus the compact data structure is defined as

$$L = (1:L_1, 2:L_2, 3:L_3, \dots, k:L_k, \dots), \quad (3)$$

$$L_k = (n_{k,1}:M(n_{k,1}), n_{k,2}:M(n_{k,2}), \dots),$$

where the label k associated with L_k refers to the triangulation k (level k), and the label $n_{k,j}$ corresponds to the number of the j th node, which was either modified in triangulation k (in the sense that its connections were different in triangulation $k - 1$) or created in triangulation k . The List $M(n_{k,j})$ is defined by (1) for node $n_{k,j}$.

Notice that for each level k , the molecular Lists are given in increasing nodal order, that is, $n_{k,1} < n_{k,2} < \dots$. It should also be pointed out that a molecular List $M(n_{k,j})$ can indeed belong to several triangulations $\tau_k, \tau_{k+1}, \dots, \tau_{k+m}$, but it only appears in the List L_k .

To illustrate the data structure defined in this section, see Figure 6 which represents the tree diagram of the triangulations (a) and (b) of Figure 5, and where the first and second branches correspond, respectively, to the first and second triangulations.

It is worth pointing out here that our molecular structure can be adequately used to store the nonzero, nondiagonal entries of the sequence of stiffness matrices associated with the triangulations (in a compact real array that uses the same pointers needed to manage the sequence of molecules). The diagonal elements can be easily managed in a separate vector.

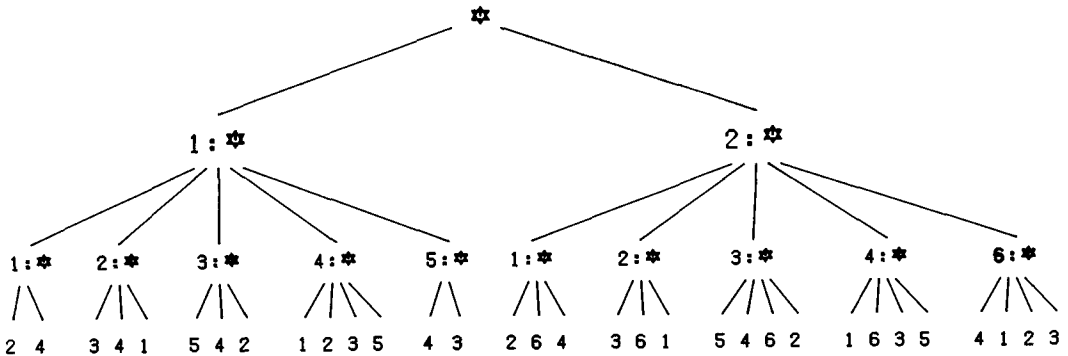


Fig. 6. Tree diagram associated with the molecular List structure of the triangulations of Fig. 5.

5.4. Machine Storage and Implementation Remarks

The ideas discussed in the preceding sections have been implemented in our experimental EXPDES package. In this section we first discuss the machine storage that this software needs to solve sparse finite-element systems associated with smooth solutions.

At this point, we have to point out that, in order to deal with a sequence of general nested triangulations, the EXPDES package uses five integer vectors: LEVEL, NMODIF, NCONEX, IPTEP, NEINOD (corresponding to the molecular List (3)), where

- LEVEL(k) is a pointer to the position where the molecules, either created or changed at level k , are stored (in increasing order) in the array NMODIF.
- NMODIF(LEVEL(K) + j) is the number of the ($j + 1$)th molecule either created or modified at level k .
- NCONEX(i) is the number of vertices of molecule NMODIF(i).
- IPTEP(i) is a pointer to the position at which the vertices of molecule NMODIF(i) are stored in the array NEINOD.
- NEINOD(IPTEP(i) + l) is the ($l + 1$)th vertex (node) of molecule NMODIF(i), with $l < \text{NCONEX}(i)$.

In other words, the molecules created or changed at level k are stored (in increasing order) between the positions LEVEL(k) and LEVEL($k + 1$) - 1 in the array NMODIF. Analogously, the vertices of molecule NMODIF(i) are stored between the positions IPTEP(i) and IPTEP($i + 1$) - 1 in the array NEINOD.

Since the global mesh-refinement algorithm of Section 4.2 is used to manage smooth solutions, the dimensions of the finite-element spaces generated increase by a factor of 2 between consecutive grids. Then, to solve a sparse linear system over a final triangulation with N vertices, approximately $2N$ molecules have to be managed in the multigrid algorithm. Since asymptotically, the average number of vertices for each molecule is 6, the basic machine storage used by the EXPDES package can be schematically represented as shown in Table I.

Table I

	Integer	Arrays	Real	Arrays
Characteristics of the nodes	IBDRY	(N)	COORX	(N)
			COORY	(N)
			U	(N)
Sequences of meshes	NMODIF	(2N)		
	NCONEX	(2N)		
	IPTER	(2N)		
	NEINOD	(12N)		
Sequences of stiffness matrices and right-hand sides			STIFF	(12N)
			DIAGNL	(2N)
			RHS	(2N)

The vectors IBDRY, COORX, COORY, and U define, respectively, the function b introduced in Section 5.2, the x coordinate, the y coordinate, and the current finite-element solution associated with the nodes. The vectors STIFF and DIAGNL and the vector RHS, respectively, contain the sequences of stiffness matrices and right-hand sides.

To ensure the clarity of the implementatin, and to make workable the full multigrid algorithm we have used $3N$ words of additional integer storage and $5N$ words of additional real storage. Then, a very conservative accounting of the machine storage needed to solve a sparse linear system over a final triangulation of N vertices gives $22N$ words of integer storage and $24N$ words of real storage.

At this point it is worth making some remarks concerning the data structure:

(i) A restricted version of this software for managing well-behaved partial differential equations with smooth solutions can certainly be developed. In such a case the full data structure is indeed more adequate. Consequently, the arrays LEVEL, NMODIF, and NCONEX and some of the work arrays become unnecessary, and the machine storage can be reduced.

(ii) The data structure we have discussed throughout this paper allows the management of general, nonsymmetric, sparse, finite-element systems. However, a symmetric version of the data structure can be developed by exploiting the characteristics of the compact row-wise representation of symmetric sparse matrices used with iterative solvers of sparse, linear systems [13]. In this case one can cut the length of the JA array (see the introduction to Section 5) from $6N$ to $3N$ locations for a mesh of N vertices by just storing ordered lists corresponding to the upper triangular part of the stiffness matrix (which also gives a columnwise accounting of the lower triangle). The "missing" vertices for some lists can be taken into account by giving each entry in JA a + or - sign, to indicate whether $(I, |JA(K)|, |JA(K+1)|)$ forms the vertices of a triangle ($JA(K) > 0$) or not ($JA(K) < 0$), where $IA(I) \leq K \leq IA(I+1) - 1$. A little reflection shows that each triangle will occur on exactly one list (that of its smallest numbered vertex) rather than three. This scheme, besides reducing the machine storage, allows the reduction of the computational work involved in the iterative sweeps over the finite-element systems, even when computing element neighbors is a little more tricky.

(iii) Here we should like to make note of the compact, reduced amount of data storage needed for our data structure in order to manage the sequence of triangulations involved in the multigrid method. In comparison, note that, in addition to the data structure used to manage the sequence of triangulations, The PLTMG package of Bank and Sherman [5, 7] uses the compact row-wise representation of sparse matrices to store the sequence of stiffness matrices. Moreover, since the machine storage required by our software increases linearly with N , the compact molecular structure compares advantageously with the structure that the classical finite-element software uses with direct solvers of sparse linear systems [10].

5.5 Implementation of the Algorithms

In this section we briefly discuss the implementation of the different algorithms combined in our software over the compact molecular List data structure we have developed.

Since this structure adequately exploits the intrinsic characteristics of sequences of nested and fully irregular triangulations, most of these algorithms can be implemented in a fairly simple way. As was pointed out in Section 5.2, the order of the molecules has been used to traverse the triangles in the global conforming mesh-refinement algorithm, in the computations of the stiffness matrices, and in the computation of the error estimators. Moreover, when local refinement is used, only the part of the stiffness matrix associated with the new and modified molecules and the error indicators for the triangles contained in these molecules are computed. Here, we should again point out that to search either a given triangle or the neighboring triangle of a given triangle, it suffices to traverse, on the average, three triangles of an adequate molecule.

The implementation of the adaptive algorithm that uses the error indicators corresponding to the triangles of the current triangulation and those corresponding to their predecessors to predict which triangles should be refined in the next triangulation has proved to be a little more complicated and expensive. More attention should be paid to this aspect of the optimization of software.

With respect to the implementation of the multigrid algorithm, the following tasks are essential:

- (i) the reconstruction of grid $j - 1$ from grid j ;
- (ii) the transfer of the residuals from grid j to grid $j - 1$.
- (iii) iterative sweeps on the Gauss-Seidel method.
- (iv) the reconstruction of grid j from grid $j - 1$.
- (v) the interpolation procedure from grid $j - 1$ to grid j .

Tasks (ii), (iii), and (v) are easily performed by working over the molecules, whereas task (iv) is quite direct, since, to reconstruct the grid j , it suffices to replace the molecules changed at level j and to include the newly created ones. In contrast, to reconstruct grid $j - 1$ from grid j , a binary search is used over the inferior levels in the array NMODIF to find the last definition of each molecule changed between levels $j - 1$ and j .

Finally, we conclude by pointing out that the dynamic multigrid iteration can be easily managed over the data structure, because each local subproblem consid-

ered is completely defined by a set of molecules contained in a contiguous block of data in the array NMODIF.

5.6. Concluding Remarks

Because of the characteristics of the multigrid method, where one needs to construct, store, and maintain the data corresponding to a sequence of finite-element spaces, it is not necessary to use processing systems to maintain the available space list, that is, to use techniques such as reference counters and garbage collection [14].

The molecular List data structure can be used to combine adaptivity with direct solvers of sparse, linear systems. In such a case it will certainly be convenient to use List processing systems to avoid the storage of superfluous information.

The complexity of implementation of our design is certainly greater than that of the classical fixed approach discussed in Section 2. However, it is a reasonable cost for obtaining a fairly general, easy to use, flexible software with more applications.

It is worth pointing out that the molecular List data structure also allows partitioning of the problem into logically coherent parts for storage in secondary devices, as well as the use of the natural parallelism and modularity of the finite-element method (see, e.g., [16, 25]). In effect, the natural parallel process structure derived from the substructure analysis used in engineering can be easily combined with our data structure.

The generalization of the data structure to three dimensions should be carefully considered, since the description of the connections of a given node is not enough to define the elements that contain the node. In the same way, the conforming mesh-refinement algorithms can be generalized for 3-simplexes by considering that in such case, sides and faces can be nonconforming.

Finally, the generalization of the molecular List data structure for managing higher finite elements can also be considered by using a parallel quasi-molecular structure containing information about the nodes that are not vertices of the triangulation.

6. NUMERICAL RESULTS

To illustrate the behavior of the EXPDES package, two numerical problems are considered. Each of these problems has been solved twice, namely,

- (i) by using the self-adaptive procedure to generate the sequence of finite-element solutions;
- (ii) by using global refinement to produce a sequence of quasiuniform triangulations.

To visualize the numerical rates of convergence obtained for each problem, we have plotted, on a log-log scale, the energy norm of the error in percentage versus the number of degrees of freedom involved, for both sequences of finite-element solutions generated. We should point out here that the slope of these curves represents the numerical rate of convergence achieved [4, 18] (the maximum rate of convergence for linear elements is represented by a slope of $-\frac{1}{2}$).

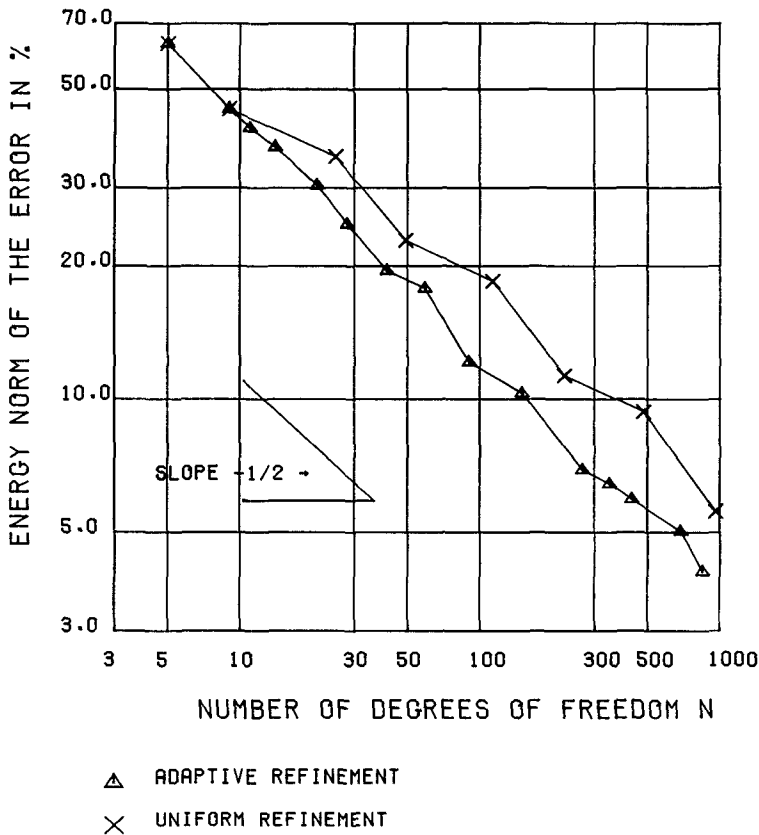


Fig. 7. Convergence behavior for the finite element solutions of Problem 1.

In order to evaluate the practical behavior of the error estimates computed, we have plotted, on a semilog scale, the effectivity index θ ($\theta = \text{estimated error/exact error}$) versus N , for each sequence of finite-element solutions generated [4, 18]. Notice that θ should tend to 1 when the error diminishes, and to be practically acceptable, $|\theta - 1|$ should be small, say, on the order of 0.2–0.3 when the relative error is on the order of 10 percent, and on the order of 0.05–0.15 when the relative error is in the range of 2–5 percent [4, 18].

6.1. Problem 1

Consider the partial differential equation

$$u_{xx} + u_{yy} - 100u = 0$$

with Dirichlet boundary conditions over the unit square [7, 17], and whose solution, $u = (\cosh(10x) + \cosh(10y))/\cosh 10$, is a nearly singular function that has a boundary layer along the sides $x = 1, y = 1$.

The convergence behavior for this problem is shown in Fig. 7, where the adaptive sequence of solutions approaches well the maximum rate of convergence for linear elements. In addition, from Fig. 8 we see that the error estimates

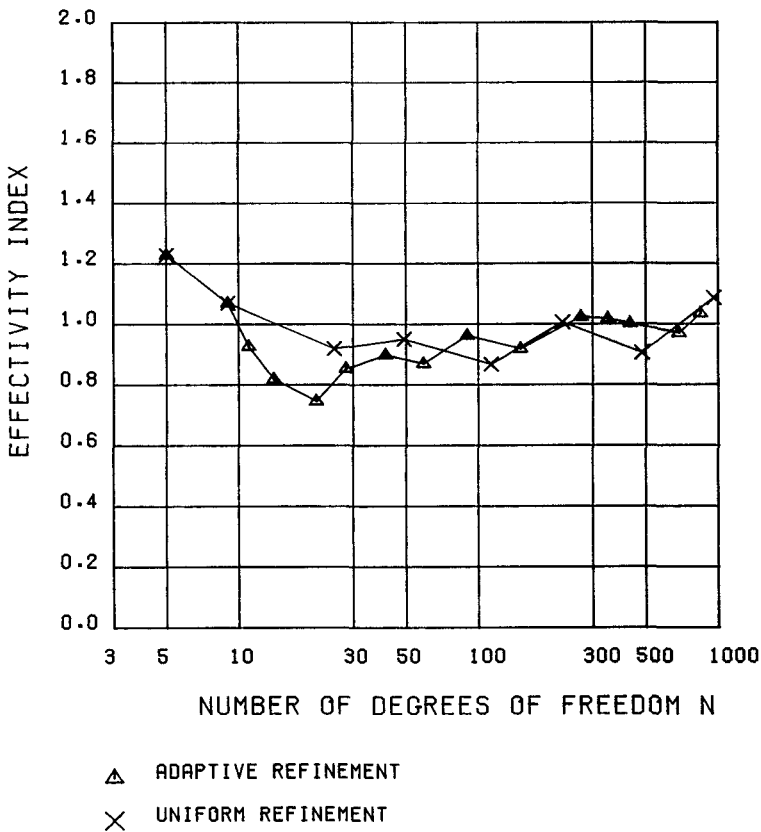


Fig. 8. Effectivity index behavior for Problem 1.

computed for both sequences of finite-element solutions, are quite acceptable from a practical point of view. Finally, to illustrate the quality of the triangulations adaptively generated, we have included in Figure 9 the last triangulation generated through the adaptive process.

6.2. Problem 2

Consider the Laplace problem with Dirichlet boundary conditions over the L -shaped region of Figure 10 [1]. The solution of this problem is the function $u = r^{2/3} \sin(2\theta/3)$, which is singular at the origin owing to the reentrant corner. Consequently, for this problem, the asymptotic rate of convergence for linear elements and quasi-uniform triangulations diminishes to $\frac{1}{3}$. However, it is known that, for an adequate non-quasi-uniform triangulation the maximum rate of convergence $\frac{1}{2}$ can again be achieved [1, 4, 18].

To solve this problem, we have used both the adaptive procedure with the dynamic multigrid algorithm and the usual global refinement procedure. The convergence behavior for this problem is shown in Figure 11, where we see that, on the one hand, the numerical rate of convergence obtained for the quasi-uniform sequence approaches the theoretical value $\frac{1}{3}$, on the other hand, the

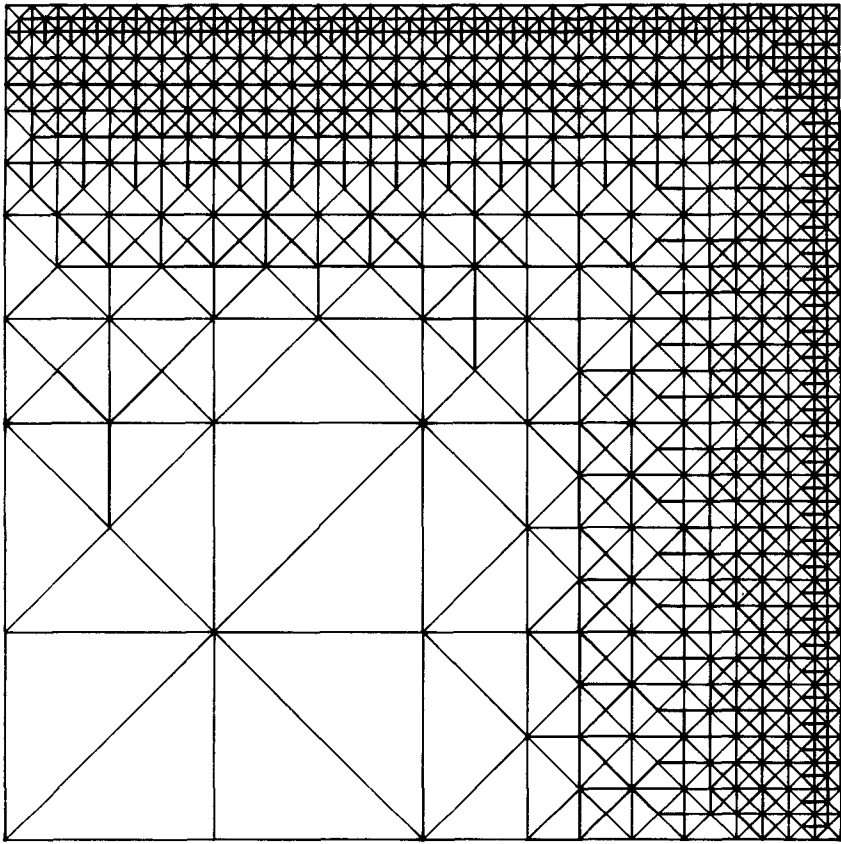


Fig 9. Last adaptive triangulation generated for Problem 1.

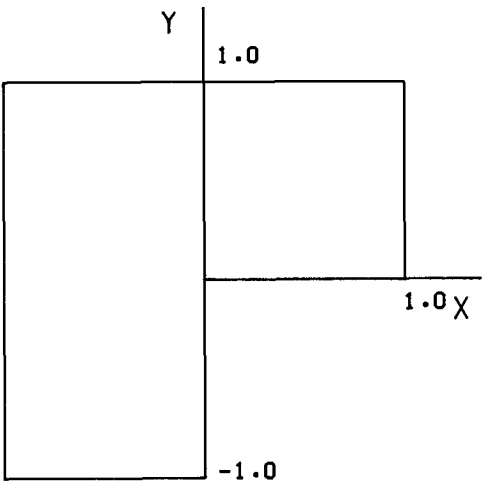


Fig. 10. L-shaped region for Problem 2.

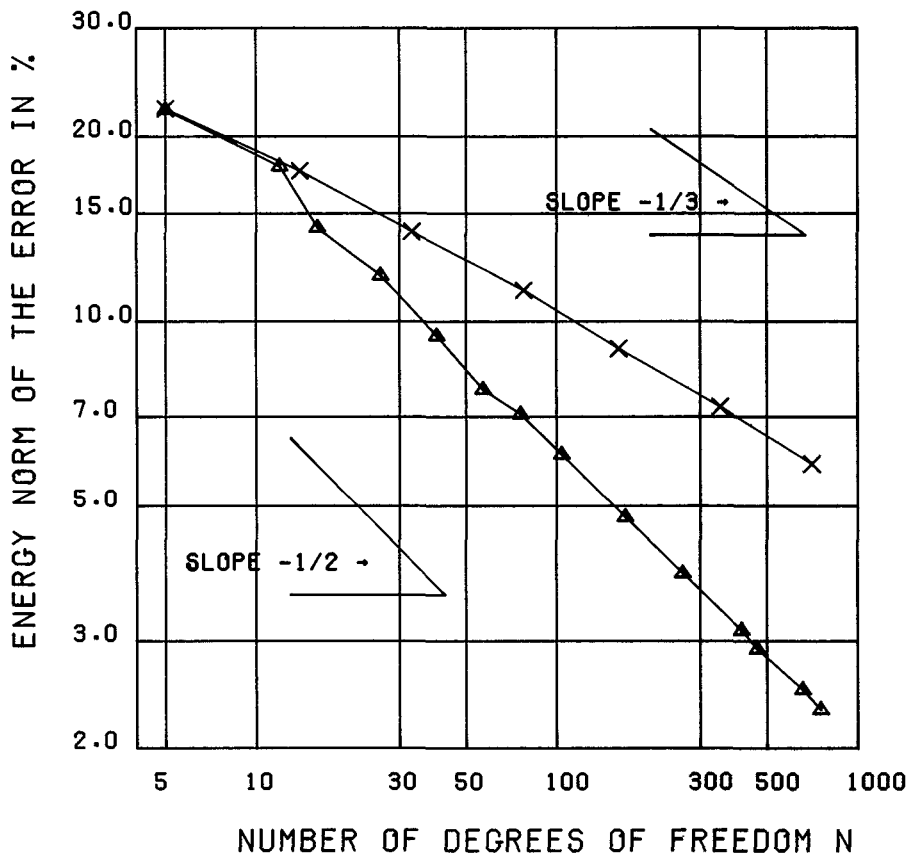


Fig. 11. Convergence behavior for the finite element solutions of Problem 2.

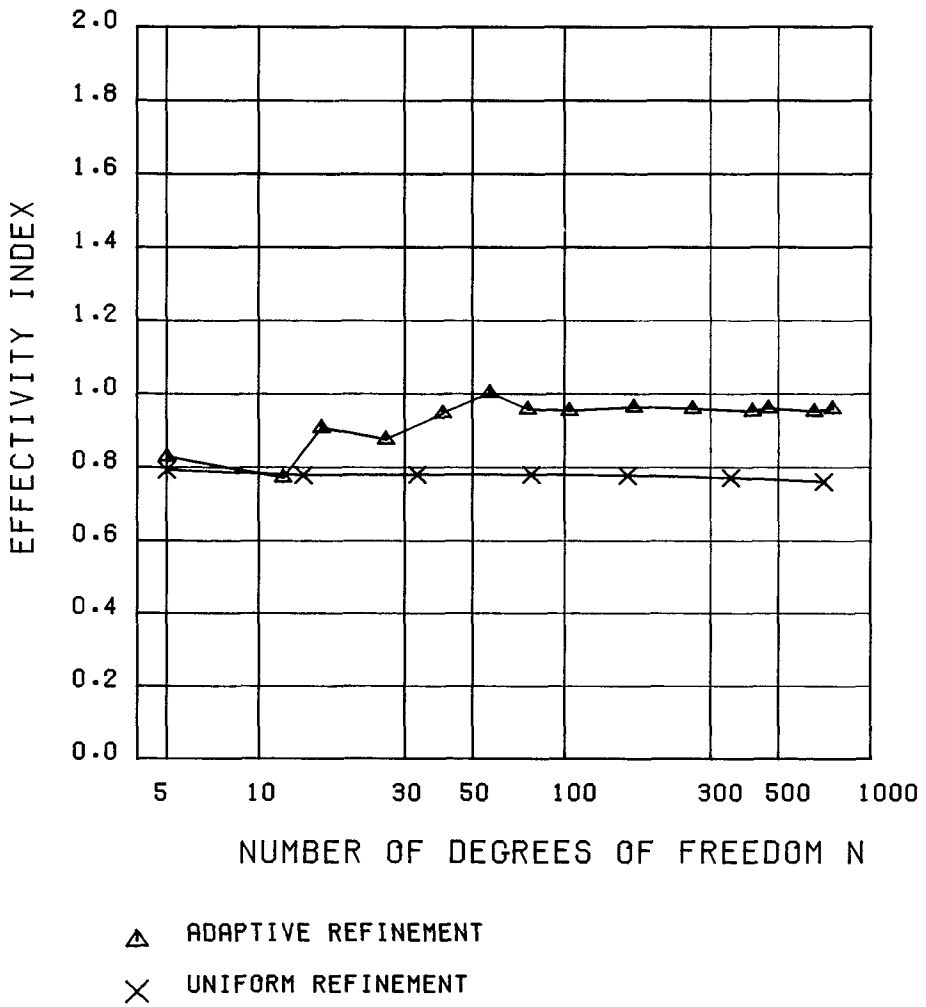


Fig. 12. Effectivity index behavior for Problem 2.

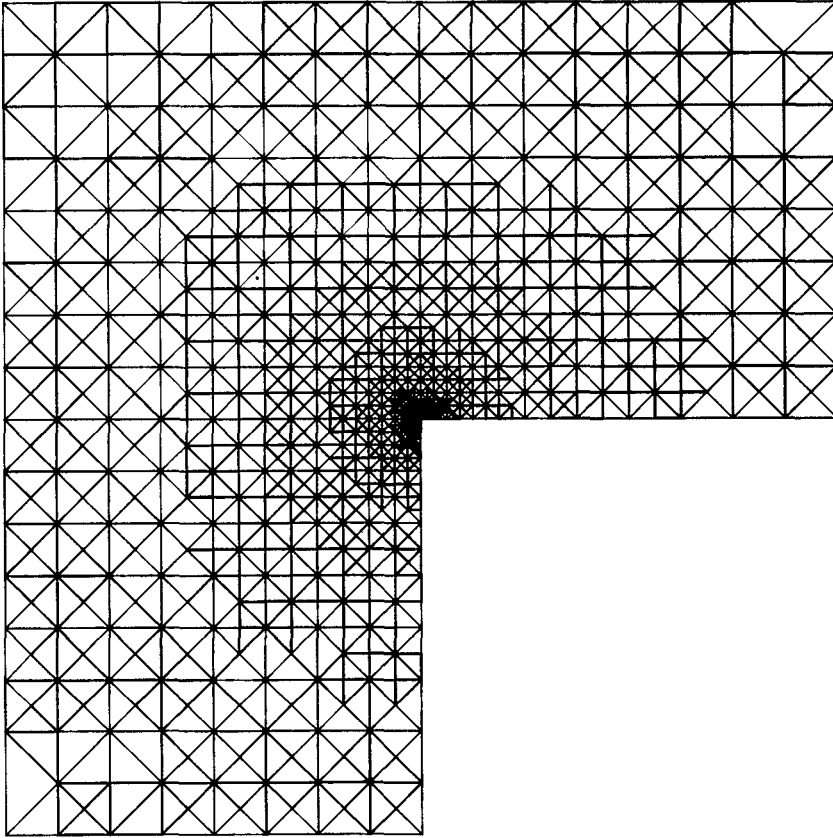


Fig. 13. Last adaptive triangulation obtained for Problem 2.

adaptive procedure has achieved the maximum rate of convergence for linear elements. In this sense we say that the adaptive sequence of finite-element solutions generated is optimal.

The effectivity index versus N has been plotted in Figure 12 for this problem. Note here the superiority of the adaptive error estimates with respect to those obtained for the quasi-uniform sequence of triangulations. However, from a practical point of view, both sequences of estimates are acceptable. Finally, to illustrate the triangulations generated, we have included in Figure 13 the last triangulation adaptively generated.

ACKNOWLEDGMENTS

I would like to thank the referee who explicitly pointed out the way of modifying the molecular List structure to manage symmetric sparse matrices.

REFERENCES

1. BABUSKA, I. The self-adaptive approach in the finite element method. In *The Mathematics of Finite Element and Applications*, J.R. Whiteman, Ed. Academic Press, New York 1976, pp. 125–143.

2. BABUSKA, I., AND RHEINBOLDT, W.C. Computational aspects of finite element method. In *Mathematical Software III*, J.R. Rice, Ed. Academic Press, New York, 1977, pp. 225-255.
3. BABUSKA, I., AND RHEINBOLDT, W.C. Error estimates for adaptive finite element computations. *SIAM J. Numer. Anal.* 15 (1978), 736-754.
4. BABUSKA, I., AND RHEINBOLDT, W.C. Reliable error estimation and mesh adaptation for the finite element method. In *Computational Methods in Nonlinear Mechanics*, J.T. Oden, Ed. Elsevier North Holland, Amsterdam, 1980. 67-108.
5. BANK, R.E. PLTMG Users' Guide. Dept. of Mathematics, Univ. of California, San Diego, June, 1981 version.
6. BANK, R.E., AND DUPONT, T. An optimal order process for solving finite element equations. *Math. Comput.*, 36 (1981), 35-51.
7. BANK, R.E., AND SHERMAN, A.H. The use of adaptive grid refinement for badly behaved elliptic partial differential equations. In *Mathematics and Computers in Simulation XXII*, North Holland, Amsterdam, 1980. 18-24.
8. BRANDT, A. Multilevel adaptive solutions to boundary value problems. *Math. Comput.* 31 (1977), 333-390.
9. GEORGE, J.A. Computer implementation of the finite element method. Ph.D. dissertation, Stanford Univ., Stanford, Calif. (1971.)
10. GEORGE, J.A. AND LIU, J.W. *Computer Solution of Large Sparse Positive Definite Systems*. Prentice-Hall, Englewood Cliffs, N.J., 1981.
11. HACKBUSCH, W. On the convergence of multigrid iterations, *Beiträge. Numer. Math.* 9 (1981), 213-239.
12. HACKBUSCH, W. Multi-grid convergence theory. In *Multigrid Methods, Lecture Notes in Mathematics* 960, W. Hackbusch and U. Trottenberg, Eds. Springer-Verlag, New York, 1982, pp. 177-219.
13. KINCAID, D.R., RESPESS, J.R., YOUNG, D.M., AND GRIMES, R.G. Algorithm 586 ITPACK 2C: A FORTRAN package for solving large sparse linear systems by adaptive iterative methods. *ACM Trans. Math. Softw.*, 8 3, (Sept. 1982), 302-322.
14. KNUTH, D.E. *The Art of Computer Programming*. Vol. 1, *Fundamental Algorithms*. Addison-Wesley, Reading, Mass., 1968.
15. NICOLAIDES, R.A. On some theoretical and practical aspects of multigrid methods. *Math. of Comput.* 33 (1979), 933-952.
16. RHEINBOLDT, W.C., AND MESZTENYI, C.K. On a data structure for adaptive finite element mesh refinements. *ACM Trans. Math. Soft.* 6 (June 1980), 166-187.
17. RICE, J.R., HOUSTIS, E.N., AND DYKSEN, W.R. A population of linear second order elliptic partial differential equations on rectangular domains. Part I. *Math. Comput.* 36 (1981), 475-484.
18. RIVARA, M.C. Adaptive multigrid software for the finite element method, Ph.D. dissertation K.U. Leuven, Belgium. 1984.
19. RIVARA, M.C. Algorithms for refining triangular grids suitable for adaptive and multigrid techniques. *Int. J. Numer. Methods Eng.* 20 (1984), 745-756.
20. RIVARA, M.C. Mesh refinement processes based on the generalized bisection of simplices. *SIAM J. Numer. Anal.* To be published.
21. SEWELL, G. A finite element program with automatic, user-controlled mesh grading. In *Advances in Computer Methods for Partial Differential Equations III*, R. Vichnevetsky and R.S. Stepleman, Eds. IMACS, Rutgers Univ., New Brunswick, N.J., 1979, pp. 8-10.
22. STUBEN, K. AND TROTTEBERG U. Multigrid methods: Fundamental algorithms, model problem analysis and applications. In *Multigrid Methods, Lecture Notes in Math.* 960, W. Hackbusch and U. Trottenberg Eds. Springer-Verlag, New York, 1982, pp. 1-176.
23. WEISER, A. Local-mesh, local-order, adaptive finite element methods with a posteriori error estimators for elliptic partial differential equations. Tech. Rep. 213. Dept. of Computer Science, Yale Univ., New Haven, CT., 1981.
24. YOUNG, D.M. Iterative solution of linear systems arising from finite element techniques. In *The Mathematics of Finite Elements and Applications*, J.R. Whiteman Ed. Academic Press, New York, 1976, pp. 439-464.
25. ZAVE, P., AND RHEINBOLDT, W.C. Design of an adaptive, parallel finite-element system. *ACM Trans. Math. Soft.*, 5 (Mar. 1978), 1-17.

Received June 1983; revised December 1983; accepted March 1984